Parte 1 — Contexto do projeto

Uma ferramenta de estudo em formato de quiz sobre redes de computadores. A ideia central não é criar uma ferramenta pensada para reforçar conteúdo de forma mais leve do que um material de estudo tradicional.

O projeto possui comentários sobre as respostas do jogador usando trocadilhos relacionados a redes. Ao final do quiz, o jogador recebe um perfil que resume como ele se saiu, e o sistema guarda um histórico de partidas, o que abre espaço para um placar comparando o desempenho de diferentes jogadores.

Por que esse projeto existe

‘ O quiz-redes é o projeto de uma bolsa de pesquisa na UFSM. A escolha do tema, redes de computadores, está ligada diretamente à área da bolsa, e o formato de quiz interativo foi escolhido como forma de tornar esse conteúdo mais acessível e menos árido do que um material de estudo convencional.

De onde veio o ponto de partida técnico

Antes do quiz-redes, já existia um projeto anterior meu chamado **`technical-service-bot`**, um bot de atendimento ao cliente escrito em Python, usando SQLite como banco de dados.

Esse projeto anterior não faz parte do quiz-redes, mas serviu como referência de estilo de código:

Os mesmos padrões de organização,
Tratamento de erro,
estrutura de banco de dados,

já tinham funcionado bem ali foram reaproveitados aqui, em vez de começar do zero decidindo tudo de novo. Isso vai aparecer repetidamente ao longo deste documento, vários dos padrões de código explicados nas próximas partes, já vinham desse projeto anterior.

Como o processo de aprendizado foi conduzido

Uma decisão importante, que vale registrar aqui porque molda o processo inteiro: o projeto não foi feito estudando teoria isolada primeiro e só depois aplicando em código. Em vez disso, cada conceito novo, banco de dados, orientação a objetos, depois desenvolvimento web, foi aprendido no momento em que ele se tornou necessário para resolver um problema real do projeto que estava sendo construído. Ou seja: o projeto cresceu por necessidade, não por um plano de estudos definido com antecedência, e o

aprendizado técnico aconteceu em paralelo, sempre ancorado em algo concreto que precisava funcionar.

Essa forma de trabalhar também define a ordem deste documento: as partes seguintes não estão organizadas do jeito que um manual organizaria. Elas seguem a ordem real em que as coisas foram construídas, o que significa que, na Parte 2, o banco de dados aparece porque foi a primeira coisa necessária para o projeto ter onde guardar informação, e os conceitos de web só aparecem bem mais à frente, no momento em que decidi migrar o quiz de um programa de terminal para um site.

Parte 2 — O banco de dados

O quiz precisa lembrar quem é o jogador, quais perguntas existem, e o resultado de cada partida jogada. Sem isso, tudo se perderia assim que o programa fosse fechado.

A escolha foi o SQLite. A diferença dele para outros bancos de dados mais conhecidos é que o SQLite não precisa de um servidor rodando separado, o banco inteiro vive dentro de um único arquivo (**`banco.db`**). Isso combinava bem com o momento do projeto: zero configuração, zero instalação de servidor, só um arquivo que o próprio Python sabe ler e escrever.

O arquivo que cria o banco: `gerar_banco.py`

Esse arquivo tem uma única responsabilidade: garantir que o banco de dados e suas tabelas existam. Ele é executado toda vez que o programa principal roda, ele foi escrito de um jeito que só cria as tabelas se elas ainda não existirem, rodar várias vezes não apaga nada nem duplica nada.

Para conversar com o SQLite, o Python usa dois conceitos que apareceram logo de início e vão se repetir no projeto inteiro:

- A **conexão**: é a ponte entre o código Python e o arquivo **`banco.db`**. Se o arquivo ainda não existe, o SQLite cria ele automaticamente ao abrir a conexão.
- O **cursor**: é o que envia comandos SQL para o banco através dessa conexão. Todo comando passa pelo cursor, nunca diretamente pela conexão.

Também logo no início do arquivo aparece uma linha.

====

cursor.execute("PRAGMA foreign_keys = ON")

====

O SQLite por padrão não obriga as regras de relacionamento entre tabelas mesmo quando essas regras são declaradas no código. Sem esse **`PRAGMA`**, seria possível, por

exemplo, salvar um resultado de partida apontando para um jogador que não existe, e o banco deixaria isso acontecer. Ligar essa opção faz o SQLite fiscalizar essas relações.

As três tabelas

O banco tem três tabelas, e a forma como elas foram divididas reflete o tipo de informação que cada uma guarda.

`jogadores;` guarda só o nome de cada jogador e a data em que ele foi cadastrado. O identificador de cada jogador é gerado automaticamente pelo próprio SQLite, incrementando um número a cada novo cadastro.

`resultados;` guarda o resultado de cada partida jogada, pontuação, número de acertos, tempo total da partida, e a data em que ela aconteceu.

Essa tabela também guarda uma referência para qual jogador jogou aquela partida, através de um campo chamado **`jogador_id`**, é justamente esse campo que é fiscalizado pelo **`PRAGMA foreign_keys`**, garantindo que todo resultado esteja de fato ligado a um jogador que existe.

Um mesmo jogador pode jogar várias partidas ao longo do tempo. Se o nome do jogador fosse repetido em cada linha de resultado, essa informação ficaria duplicada, a divisão das tabelas corrige isso, e o PRAGMA contra a relação das chaves estrangeiras.

`perguntas;` guarda o banco de questões do quiz, o enunciado, as quatro alternativas, qual delas é a correta, a categoria e o nível de dificuldade.

Diferente das outras duas tabelas, não há relações declaradas, quando uma partida é salva, o sistema guarda apenas o resultado agregado daquela partida, e não registra quais perguntas específicas o jogador acertou ou errou.

Essa foi uma escolha para manter o escopo do projeto simples, se no futuro fosse necessário saber exatamente quais perguntas cada jogador errou, seria preciso criar uma tabela nova conectando **`resultados`** e **`perguntas`**, mas isso não foi necessário até este momento do desenvolvimento.

Encerrando a operação no banco

No fim do arquivo, duas chamadas garantem que tudo funcione corretamente:

`====`

```
conexao.commit()
conexao.close()
```

`====`

O **`commit()`** é necessário porque o SQLite trabalha com o conceito de transação: as mudanças feitas, comandos rodados durante o código ficam pendentes até serem

confirmadas com esse comando, é o **`commit()`** que grava tudo no arquivo. Já o **`close()`** fecha a conexão, liberando o arquivo do banco.

Esse mesmo padrão — abrir conexão, pegar cursor, executar comando, dar commit, fechar conexão — se repete em praticamente todo lugar do projeto que precisa conversar com o banco de dados, como vai aparecer na próxima parte.

Parte 3 — As classes `Jogador` e `Pergunta`

Com o banco criado, o próximo problema foi representar um jogador ou uma pergunta dentro do código Python, sem ficar escrevendo SQL solto espalhado pelo projeto toda vez que precisar salvar ou buscar algo. As classes **`Jogador`** e **`Pergunta`,** cada uma responsável por saber como se salvar e se buscar no banco.

O construtor e o `self`

T Toda classe em Python tem um método **`__init__`**, que roda automaticamente toda vez que um objeto novo é criado. No caso de **`Jogador`**:

=====

```
def __init__(self, nome):
    self.nome = nome
```

====

O **`self`** aqui representa o próprio objeto que está sendo criado. Se eu crio dois jogadores diferentes, **`Jogador("Teteu")`** e **`Jogador("Maria")`**, cada um guarda seu próprio **`self.nome`**, são objetos separados na memória, mesmo usando o mesmo código de classe.

Toda vez que um método da classe é chamado a partir de um objeto, o Python passa esse objeto automaticamente como **`self`**, por baixo dos panos, é assim que o método sabe de qual jogador especificamente ele deve tratar.

Salvando um jogador no banco

====

```
def salvar_no_banco(self):
    ...
    cursor.execute("""
        INSERT INTO jogadores (nome)
        VALUES (?)
    """, (self.nome,))
```

====

Um detalhe de segurança importante, o valor não é inserido direto na string do SQL, e sim passado separadamente através do **?** - **query parametrizada**.

Se o nome do jogador fosse inserido diretamente dentro do texto do comando SQL, um nome malicioso poderia alterar o comando e comprometer o banco - **SQL Injection**. Usando **?**, o valor é sempre tratado como dado puro, não como parte do comando.

Depois do **`INSERT`**, o código pega o identificador que o SQLite acabou de gerar automaticamente:

====

id_gerado = cursor.lastrowid

====

Esse identificador é o que vai ser usado depois para ligar um resultado de partida a esse jogador específico, na tabela **`resultados`**.

`@staticmethod`: métodos que não precisam de um jogador específico

Nem todo método da classe **`Jogador`** depende de um objeto já criado. **`salvar_resultado`**, por exemplo, recebe tudo que precisa através de parâmetros.

====

@staticmethod
def salvar_resultado(jogador_id, pontuacao, acertos, tempo_total):

====

Esse método nunca usa **`self`**, porque ele não olha para nenhum atributo de um jogador específico, ele só recebe números e um id, e insere isso na tabela.

Por isso ele é marcado como **`@staticmethod`**, que é um modo estático, isso permite chamá-lo direto pela classe, sem precisar ter um objeto criado antes.

Tratamento de erro

Os métodos que mexem no banco seguem o mesmo padrão: **`try/except/finally`**. O **`try`** tenta executar a operação; o **`except`** captura erros específicos separadamente de erros genéricos; e o **`finally`** garante que a conexão com o banco seja sempre fechada no final, mesmo que algo dê errado no meio do caminho, isso evita que o arquivo do banco fique travado.

A classe `Pergunta` e a tradução entre banco e código

`Pergunta` segue a mesma lógica de **`Jogador`**, mas expõe um problema novo, no banco, as quatro alternativas de uma pergunta são quatro colunas separadas. Em Python, porém, trabalha-se com essas quatro alternativas como uma lista só.

====

self.alternativas = alternativas # ["Ethernet", "Wi-Fi", "Fibra Óptica", "Bluetooth"]

====

Isso cria a necessidade de uma tradução em dois sentidos.

Ao salvar, a lista é desmontada em quatro posições separadas, na ordem certa, para bater com as quatro colunas do **`INSERT`**:

====

(self.enunciado, self.alternativas[0], self.alternativas[1],
 self.alternativas[2], self.alternativas[3], self.resposta_correta,
 self.categoria, self.dificuldade)

====

Ao buscar, o caminho é o inverso. O SQLite devolve cada linha da tabela como uma tupla crua, só valores, na ordem das colunas, sem nome nenhum ligado a cada posição:

====

cursor.execute("SELECT * FROM perguntas")
linhas = cursor.fetchall()

====

E cada tupla crua é reconstruída manualmente em um objeto **`Pergunta`**, remontando as quatro colunas separadas de volta em uma lista,

====

pergunta = Pergunta(
    enunciado=linha[1],
    alternativas=[linha[2], linha[3], linha[4], linha[5]],
    resposta_correta=linha[6],
    categoria=linha[7],
    dificuldade=linha[8]
)

====

Desmontar objeto em colunas para salvar;
Remontar colunas em objeto;
Acontece porque o banco de dados e o Python entendem estrutura de dados de formas diferentes, o SQLite só sabe lidar com linhas e colunas fixas, enquanto o Python

trabalha melhor com objetos e listas. e nesse projeto essa tradução é feita manualmente, dentro das próprias classes.